Full Stack App Development Evaluation and Analysis Report

1. Domain Context & FYP Clarification

This prototype is an independent system and does not share the scope of the Final Year Project (FYP). It was developed specifically to demonstrate core full-stack competencies: decoupled component architecture, asynchronous reactive streams, and RESTful ORM persistence. These architectural patterns provide a transferable baseline for enterprise software development, including the author's FYP.

2. Backend REST API Design

2.1 Endpoint Structure

The REST API uses pluralized resource nouns and standard HTTP verbs:

- GET /api/books: Fetches all books (supports sorting and category filtering).
- GET /api/books/{id}: Fetches a single book by identifier.
- POST /api/books: Validates and persists a new book record.
- PUT /api/books/{id}: Updates an existing record deterministically.
- DELETE /api/books/{id}: Removes a book record by identifier.
- GET /api/books/search: Performs custom JPQL keyword search.

2.2 Path Variables vs. Query Parameters

- Path Parameters (/{id}): Identifies a discrete, unique entity via its primary key.
- Query Parameters (?category=...&sortBy=...): Used for filtering, sorting, and pagination without altering the base resource URL, keeping filtered views shareable and bookmarkable.

3. Frontend Architecture & Routing

3.1 Component Hierarchy

A strict three-tier hierarchy enforces separation of concerns:

- Root (AppComponent): Global application shell and navigation bar.
- Smart Container (BookListComponent): Orchestrates HTTP requests, manages state, reads route params, and coordinates delete/filter actions.

- Dumb Child (BookCardComponent): Agnostic to HTTP services; receives data via @Input() and emits actions (edit, delete, toggleStatus) via @Output().
- Leaf Grandchild (BookStatusBadgeComponent): Visual presentational badge displaying dynamic styles based on book availability.

3.2 Strategic Placement of HTTP Calls

HTTP calls are strictly centralized in container components via an injectable singleton (BookService). Dumb child components never inject HttpClient. This guarantees a single source of truth, prevents duplicate network calls, and enforces unidirectional data flow (data down via inputs, events up via outputs).

3.3 Routing Strategy (app.routes.ts)

- Explicit routes (books, books/new, books/edit/:id) separate browsing from state changes.
- Default redirection ('' -> 'books') and wildcard fallback (**) ensure robust navigation.
- Programmatic navigation (Router.navigate()) redirects users to the catalog after form submissions.

4. Directives, Bindings & Reactive Forms

4.1 Bindings & Modern Control Flow

- Interpolation & Property Binding: {{ }} renders text dynamically; [disabled] manages button states; (click) and (ngSubmit) capture user events.
- Built-in Control Flow (@if, @for): Replaces legacy *ngIf/*ngFor. @if/@else handles conditional empty states. @for utilizes tracking keys (track book.id) and contextual variables ($index, $count, $first) to display total records and identify top results.
- Dynamic Styling ([ngClass], [ngStyle]): Dynamically applies contextual badge styles and indicator borders based on real-time availability.

4.2 Reactive Forms Design & Validation

Reactive Forms (FormGroup, FormControl) were chosen over template-driven forms for programmatic integrity:

- Direct Model Mirroring: The client schema directly mirrors validation rules on the backend entity (Book.java).
- Validation Rules: Enforces mandatory fields (Validators.required), correct ISBN-10/13 format via regex (Validators.pattern), and valid publication year boundaries (min(1000), max(2030)).
- UX Feedback: Errors only display when controls are dirty/touched (f['title'].touched && f['title'].invalid), preventing premature validation warnings, while [disabled]="bookForm.invalid" blocks malformed submissions.

5. Asynchronous Network & UI Integration

5.1 Asynchronous HTTP via RxJS

All HTTP calls return RxJS Observable streams:

- Prevents the UI thread from blocking during backend database queries.
- Lifecycle cleanup (Subscription.unsubscribe() in ngOnDestroy) prevents client-side memory leaks.
- Enables composability with RxJS operators (catchError, map) for future error handling or data transformations.

5.2 Facilitating Backend Business Logic for the End-User

- User actions (e.g., toggling availability or submitting updates) dispatch background HTTP requests and update the UI upon receiving HTTP 200 or 204 responses, eliminating full page reloads.
- Backend errors caught by Spring Boot's @RestControllerAdvice return structured JSON payloads with precise HTTP status codes (e.g., 400 Bad Request, 404 Not Found), allowing the UI to present actionable field-level errors rather than unhandled crashes.

6. Conclusion

The system implements a production-ready full-stack architecture that fulfills all rubric requirements. Decoupling an Angular reactive UI from a Spring Boot REST service ensures high maintainability, predictable data flow, and responsive user feedback.